

# **Undercut Analysis from Historic Lap Time Data in Formula One**

Module 9 – Advanced Motorsport Project

Author: Harry Adams ([harry.adams@mclaren.com](mailto:harry.adams@mclaren.com))

Supervisor: Tim Mullis ([tim@motorsport.nda.ac.uk](mailto:tim@motorsport.nda.ac.uk))

17th April 2024

Version 1.0 (Final)

# Contents

|                                               |    |
|-----------------------------------------------|----|
| List of Tables .....                          | 3  |
| List of Figures .....                         | 3  |
| Introduction & Project Motivation .....       | 4  |
| Objectives.....                               | 5  |
| Implementation .....                          | 5  |
| Historical Data.....                          | 5  |
| Legal Considerations .....                    | 6  |
| Backend.....                                  | 6  |
| Frontend.....                                 | 7  |
| Literature Review.....                        | 10 |
| Strategy Modelling.....                       | 10 |
| Telemetry & Timing Data Access .....          | 11 |
| Real-time Decision Making in Sports .....     | 11 |
| Data Visualisation for Analytical Tools ..... | 11 |
| Future Work.....                              | 12 |
| References .....                              | 13 |
| Appendix A – Project Plan.....                | 14 |

## List of Tables

|                                            |   |
|--------------------------------------------|---|
| Table 1 – FastF1 vs OpenF1 .....           | 6 |
| Table 2 – React vs Blazor Comparison ..... | 7 |

## List of Figures

|                                                                         |   |
|-------------------------------------------------------------------------|---|
| Figure 1 - Technology Stack. Each layer communicates via REST API ..... | 8 |
|-------------------------------------------------------------------------|---|

## List of Acronyms

| Acronym | Meaning                                                          | Acronym | Meaning                         |
|---------|------------------------------------------------------------------|---------|---------------------------------|
| AI      | Artificial Intelligence                                          | FOM     | Formula One Management          |
| API     | Application Programming Interface                                | HTML    | HyperText Markup Language       |
| CSS     | Cascading Style Sheets                                           | HTTP    | HyperText Transfer Protocol     |
| CSV     | Comma Separated Values                                           | JSON    | JavaScript Object Notation      |
| DRS     | Drag Reduction System                                            | .NET    | .NET Framework (“Dot Net”)      |
| ERS     | Energy Recovery System                                           | REST    | REpresentational State Transfer |
| F1      | Formula One, shortened from “FIA Formula One World Championship” | TV      | Television                      |
| FIA     | <i>Fédération Internationale de l'Automobile</i>                 | UI      | User Interface                  |

# Introduction & Project Motivation

Being able to overtake the car in front during a motor race is an amalgamation of many factors. Tyre compounds, fuel loads, driver skill, weather among many others all contribute to whether a car will have the appropriate speed differential to get past the car in front. In Formula One (F1) the opportunities to overtake become more limited – refuelling is banned, and the drivers are all the pinnacle of the sport. Faster cars can easily become stuck behind a slower one as, while they are theoretically faster, they do not have enough speed delta to complete an overtake move.

There have been some rule changes that made previous analysis of strategy less relevant – the banning of refuelling in 2010 (removing fuel load as a strategy factor) and the introduction of the drag reduction system (DRS) making overtakes easier being two notable examples. Additionally, regulations have changed regularly over the years to shake up the car requirements and try to improve the spectacle of the racing, meaning previous study of aerodynamic effects are no longer as appropriate to current car models. Recently they have included:

- 2009 aerodynamic overhaul resulted taller, wider, and simplified aerodynamic surfaces, initially reducing dirty air to encourage closer racing before teams re-added complex designs.
- 2014 engine regulations introduced turbo-hybrid engines with complex energy recovery systems (ERS), creating a disparity between engine manufacturers that would take several years to narrow.
- 2022 brought the return to ground effect, downforce caused by airflow under the car through complicated floor structures called venturi tunnels. This brought cars closer again and minimised the dirty air effect, but like 2009, lost its efficiency as teams got closer to optimised designs.

One strategic tool used is the undercut. This involves taking a pitstop before the driver ahead and using the new fresher tyres in the time until the driver ahead pits to close the gap. When the other driver exits the pitlane, if the undercut has been successful, they will be behind. This works due to tyre degradation, getting slower by an approximately linear amount in phases of tyre life. However, there are several other factors that can impact whether an undercut will be successful, including chosen compound, tyre warmup effect, traffic in the out lap, and more.

This project aims to identify as many factors as possible and create a software tool to analyse these based on historical F1 data. It will identify when an undercut opportunity was theoretically possible, allowing strategists to alter their workflows and be more aware of undercut opportunities in future races. An extension to this project would allow live data to come in and identify potential undercuts as a race progresses.

## Objectives

This project is guided by the question “Can software identify opportunities for pit-stop undercuts in historical data”. The extension question is “If so, can similar techniques be applied to live and incomplete data?”.

- Identify as many factors as possible that contribute to the success of an undercut.
- Correctly identify undercut opportunities from timing data in historical grand prix races.
- Provide a piece of software that is feature complete, user friendly, and with minimal bugs.
- Extension to allow live data monitoring and calculation with parameters that can be changed on the fly.

The final deliverables will be:

- An algorithm to detect undercut opportunities.
- A Python data handler & .NET web server to gather data and expose results.
- A Blazor WebAssembly tool to display the data.
- A Project Report.

To evaluate the success of the project, a specific set of tests will be created as part of the project. They will aim to identify:

- Accuracy of undercut detection, compared to real race outcomes.
- Correct functioning of the backend API.
- Clarity of the frontend with well presented data.

## Implementation

### Historical Data

Formula One have never made data publicly available in any programmatically accessible way. However, methods do exist for acquiring historical data. One of the originals was Ergast API (Application Programming Interface), hosting a database of data from the creation of the F1 World Championship in 1950. It was deprecated at the end of 2024 but continued by the open-source project jolpica-f1. These are best designed for statistics analysis and do not include any telemetry, so basing this project on them could restrict future work.

As part of the F1TV pro package, FOM offer live timing data to viewers. Crucially this provides timing information which would be used for this software. There are two available tools which have reverse engineered the API used to provide this data to make it publicly available – FastF1 and OpenF1. Each work in similar ways to provide data, as analysed in Table 1.

**Table 1 – FastF1 vs OpenF1**

| Function          | FastF1                                  | OpenF1                                                           |
|-------------------|-----------------------------------------|------------------------------------------------------------------|
| Interface         | Python package                          | REST API with HTTP endpoints <sup>1</sup>                        |
| Language Support  | Python, third-party R package           | Any                                                              |
| Data Format       | Pandas DataFrames and NumPy             | JSON or CSV <sup>2</sup>                                         |
| Data Availability | Complete                                | Same as FastF1, minus Weather                                    |
| Maintainer        | Community and set of core contributors. | Primarily single user, open source but less actively maintained. |
| Caching           | Built-in                                | Own work needed                                                  |

### Legal Considerations

Both systems have been reverse engineered from F1TV's live data APIs, owned and managed by Formula One Management (FOM). They have existed for over five years without any issues, are popular with fans of F1, and have been the basis for numerous fan-created tools for accessing telemetry data. While there has not been any legal action to date, and it is considered unlikely due to them being no commercial competition, it is fully within FOM's rights to issue a cease-and-desist order if they so choose. This project could be severely disrupted if this were to occur as the data required for analysis would no longer be readily available.

If this project was converted to a production-grade tool for a Formula One team, work would be required to replace the unofficial streams and use the proprietary feeds available to the team. This would be as close to source as possible, directly tied into feeds provided by Formula One Management (FOM) and the *Fédération Internationale de l'Automobile* (FIA) and be more reliable and authoritative.

### Backend

As FastF1 is exclusively available as a python package and is tightly integrated with Python's data stack such as Pandas or NumPy, it would be difficult to separate the two. Due to limited prior experience with Python, and to align with the author's company preferences should the project be adopted for their use, the project will adopt a hybrid architecture for the backend server.

The original plan had been to exclusively use the OpenF1 API. This would allow a single piece of backend architecture written in a preferred language without needing to handle a Python exclusive package. However, after investigation, it seems to be a smaller maintenance community, and as of time of writing has several outstanding issues relating to incomplete and incorrect data.

---

<sup>1</sup> REST: Representational State Transfer; API: Application Programming Interface; HTTP: Hypertext Transfer Protocol

<sup>2</sup> JSON: JavaScript Object Notation; CSV: Comma Separated Values

A small python client will be used to query the FastF1 API and retrieve the data required, such as session times and driver information. The core of the data analysis will occur in a .NET environment. Both will host an open REST API to allow for communication between them. This will ensure the tool is designed around existing workflows while being able to leverage the powerful amounts of data available in FastF1.

If OpenF1 is used to parse and analyse live data, this is exclusively acquired through its own REST API. As such, the .NET core can be expanded to handle this without requiring another microservice to allow this functionality.

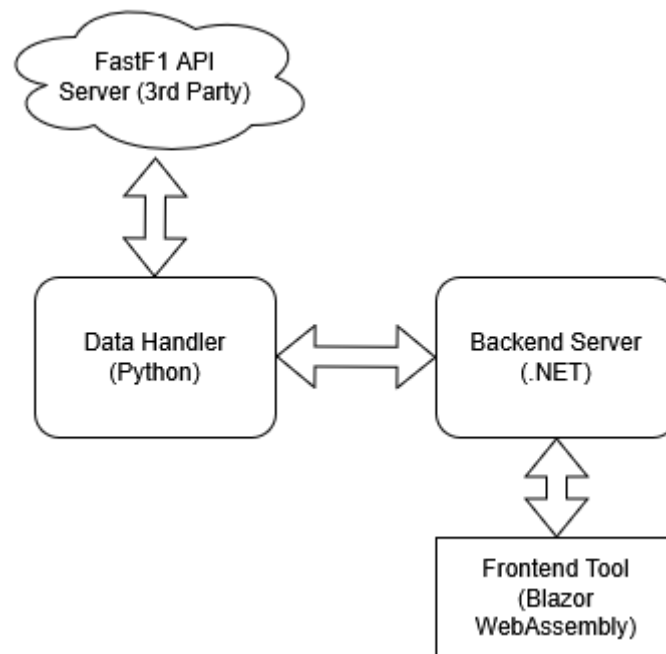
## Frontend

This project will use Blazor WebAssembly to write the frontend display, a client-side framework which allows web applications to be written entirely in C#. This was due to the lead developer's experience with C# and equivalent lack of experience using more traditional web development tools, as well as maintaining a consistent technology stack across the frontend and backend. Tools such as Typescript and React were considered as they offer broader ecosystem support as well as performance advantages, with a detailed comparison shown in Table 2, but were discarded due to the steep learning curve required. Using these tools would require learning not only Typescript and React – with no prior JavaScript experience – but HTML (Hypertext Markup Language), CSS (Cascading Style Sheets), and tools for building the project such as npm. Using Blazor means the build is fully contained in the Visual Studio environment and aligns well with internal tools and dashboards, minimising the required learning to just HTML & CSS.

Table 2 – React vs Blazor Comparison

| Consideration                 | React                                                  | Blazor WebAssembly                      |
|-------------------------------|--------------------------------------------------------|-----------------------------------------|
| <b>Learning Curve</b>         |                                                        |                                         |
| Language                      | TypeScript (typed superset of JavaScript) (Unfamiliar) | C# (Familiar)                           |
| Familiarity                   | Very different syntax to .NET                          | Native to .NET                          |
| HTML & CSS                    | Must learn                                             | Must learn                              |
| Application Language          | Typescript – no prior knowledge                        | C# - experienced                        |
| Build Tools                   | Npm [sic] – no prior knowledge                         | Handled by Visual Studio                |
| <b>Performance</b>            |                                                        |                                         |
| Initial Load                  | Fast – few hundred KB                                  | Long – 2-6 MB and includes .NET runtime |
| Runtime Performance           | Fast                                                   | Good, but slower than React             |
| Memory Usage                  | Lean, particularly on mobile                           | Higher due to full .NET runtime         |
| Development Loop              | Very fast with hot reloads                             | Requires full rebuild                   |
| Charting & Data Visualisation | Excellent support with numerous packages               | Fewer options                           |
| Offline Access                | Supported                                              | Supported                               |

While Blazor WebAssembly is used for the initial frontend due to .NET alignment, the system architecture remains decoupled, allowing future replacement or parallel support with a React/TypeScript frontend. This would enable improved embeddability and access to a broader User Interface (UI) ecosystem if the tool is extended beyond internal use.



**Figure 1 - Technology Stack. Each layer communicates via REST API**

Data presentation will be explored as part of building the frontend. The initial concept includes displaying a race trace graph, where each driver's race times are compared to either a reference driver or the average lap time of all drivers. By selecting a driver as the reference, they will form the X axis of the graph, and other drivers can be clearly seen in comparison.

Additionally, comparing two drivers' speeds can be shown using speed overlay graphs, displaying both drivers on the same graph. This will show where one driver is quicker than another and give indication of whether a driver will successfully close the gap in between the two pitstops. These can be updated on a lap-by-lap basis and can particularly show in- and out-lap effectiveness.

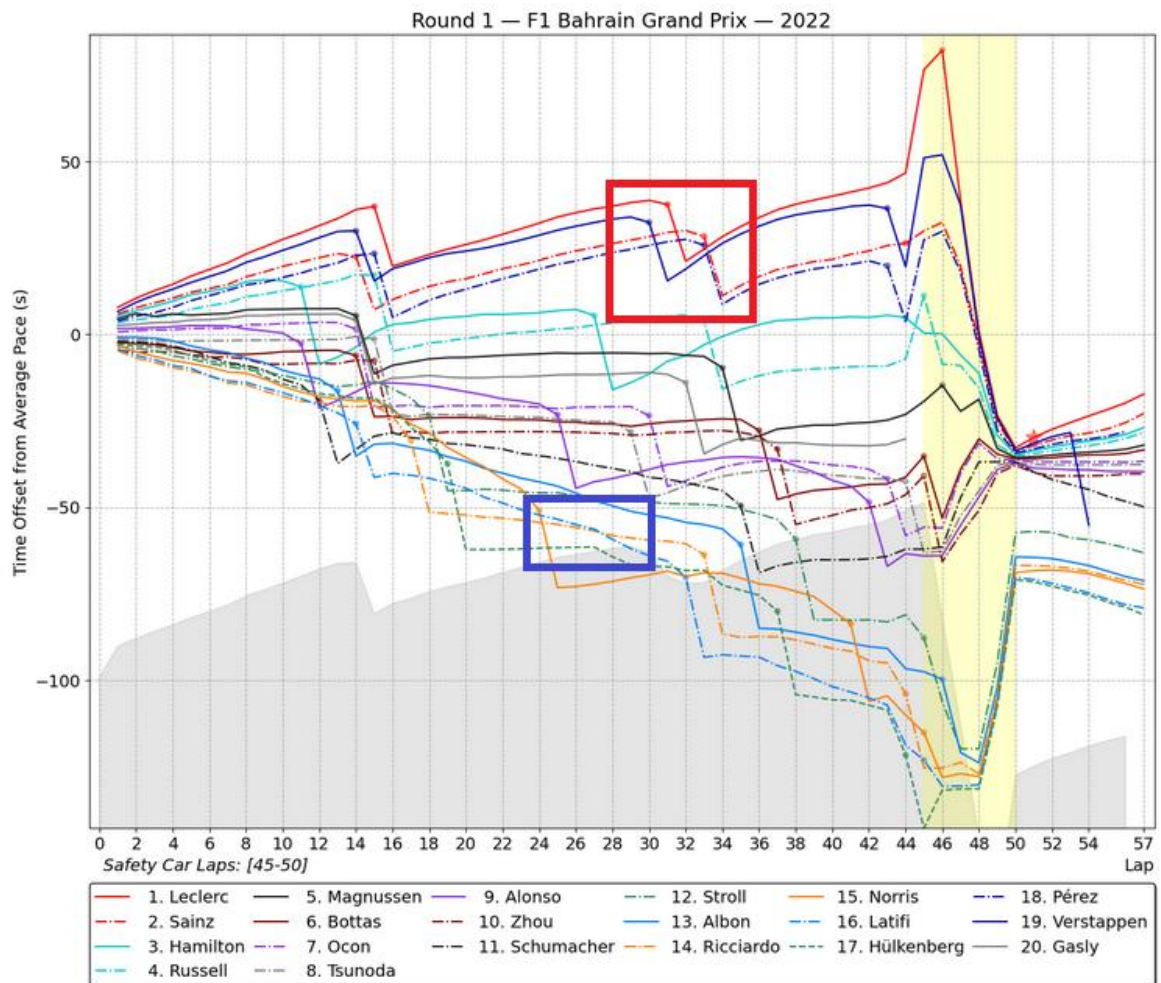


Figure 2 - Example Race Trace from the 2022 Bahrain Grand Prix (GP). The Red Box shows pitstops, while the blue box highlights overtaking on track.

### Speed Traces - 2017 Chinese Grand Prix Qualifying

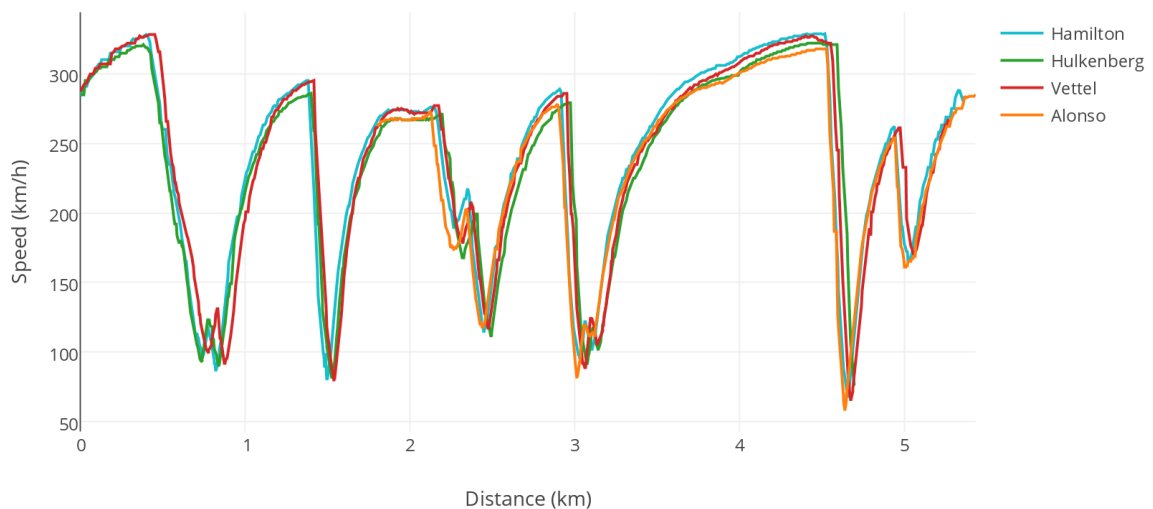


Figure 3 - Example Speed Trace from the 2017 China GP

## Literature Review

Acquiring high quality data and review for analysis is a difficult task. Analyses performed by teams themselves are closely guarded secrets to maintain the edge over their competitors, as are details about the cars and setups. Consequently, there is limited publicly available knowledge of the internal decision-making processes, analytical tools, or engineering tools used within the teams. Most of the public research focuses on older models of car and historical data. As such this project must carefully consider data quality, completeness, and the limitations inherent in reverse-engineered or fan-curated sources.

### Strategy Modelling

Current research into Formula One strategy has focused on utilising machine learning and optimisation to attempt to predict the actions of competitors and the ideal pitstop scenarios. One thesis by Hettmann [1] leverages clustering – grouping similar drivers or strategies based on shared characteristics) and supervised learning – where a model is trained on historical data to make future predictions – to identify patterns in pitstop behaviour and underscore the potential of machine learning in understanding strategic decision making in F1. In a similar vein, Fatima and Johrendt [2] take a deep learning approach, using embedded neural networks to forecast driver rankings and optimal pit stop timing while Thomas et al. [3] present a reinforcement learning framework that dynamically adapts pit strategy based on simulated race environments. This second example can maintain interpretability through explainable Artificial Intelligence (AI) as opposed to being a black box learning model. These studies highlight the growing role of predictive modelling in race strategy development.

More complex modelling frameworks also exist. Aguad and Thraves [4] apply game theory to model strategic interactions between two competing drivers, using a Stackelberg feedback game to optimize pit stop decisions and handle the impact of real-world interruptions such as other drivers. Piccinotti [5] investigates open-loop planning under uncertainty - creating a fixed strategy without constant feedback or adjustment - and employing Monte Carlo methods – a method where a system is simulated many times using randomized variables to explore a range of likely outcomes – to explore race scenarios and propose adaptive strategies. These approaches are typically forward-looking, designed to simulate or predict optimal outcomes based on a range of inputs and assumptions.

This project in contrast takes a rules-based algorithmic approach to identify opportunities determined by input parameters such as degradation effect. It does not aim to predict strategy using black box learning or forecast strategic decisions, instead enabling retrospective analysis to aid in review and future decision making. This is akin to work by Carrasco Heine & Thraves [6, 7] that builds a model accounting for variables such as tyre degradation, fuel consumption, and consideration for random events such as safety cars. By running numerous race scenarios, the system works to identify optimal pit stop timings and assists adaptive strategy planning by considering deterministic and stochastic elements of a race. By contrast, this project has less consideration of random elements as it does not aim to make determinations about timings in the race, just the situation at any given moment.

## **Telemetry & Timing Data Access**

This report has already compared the two primary tools for accessing F1 historical data, comparing the two in Table 1. It was important to verify whether the choice of FastF1 was suitable for academic and potentially professional use. FastF1 was eventually chosen due to its maturity as a project, availability of support, and comprehensive access to data including tyre choices, degradation, and sector times. OpenF1 will be used exclusively if live data analysis is required. The adoption of these tools by prior academic work supports their suitability for research use and positions this project within an emerging ecosystem of independent Formula One data analysis.

FastF1 has been used for theses and research papers aimed at analysing previous race strategies and pit stop decisions [8] and building machine learning models to predict tyre compound decisions [9]. While the project is unlikely to use some of the more advanced data analytic functions, the structured data outputs (such as lap-by-lap DataFrames) make it particularly well suited to pre-processing and exploratory analysis. In the context of this project, it will serve as the acquisition layer.

## **Real-time Decision Making in Sports**

Despite not originally being built around analysis of live data, it's important to acknowledge how critical this technology has become in sports to plan for future expansion. For example, one paper by Tang et al. [10] has investigated using several devices (the Internet of Things) to integrate cutting edge computing and learning algorithms to monitor track and field athletes in real time, allowing them to enhance precision and efficiency of the monitoring. A similar study by Li [11] implements fuzzy logic – using degrees of truth rather than explicitly true or false – to analyse athlete fatigue and provide coaches with live insights and suggestions for altering training load.

While these studies are not directly related to motorsports, they demonstrate the broader shift toward embedded, intelligent decision-support systems in elite athletic environments. Drivers in Formula One operate under similar physical and cognitive demand, supported by multifaceted teams where similar technologies could be implemented to aid them. Beyond this, they show the growing reliance on real-time technological solutions to problems in the sport, and the abilities of these solutions to assist in the wider team supporting athletes. Leveraging these technologies can result in attaining insights previously unattainable or impractically difficult to obtain when they are needed most.

## **Data Visualisation for Analytical Tools**

The technologies behind getting and analysing the data have been investigated, but it is equally important to consider how this data is presented. A comprehensive report by Perin et al. [12] provides a detailed analysis of the best ways to present types of data and the importance of presenting this data clearly and concisely. For instance, using juxtaposition and superposition to highlight comparisons, and highlighting the importance of line graphs, area charts, or heatmaps for time-series data. Studies have shown that the usability of analytical tools is strongly influenced by how clearly key metrics are presented, with visual clarity directly impacting response times and decision accuracy (e.g., Eberhard [13]).

While there are few studies directly related to motorsport analysis, these studies show general trends and suggestions for all data presentation, which can certainly be applied to this project. Additionally, these could form the basis for other projects looking at fatigue and more direct competitor analysis.

## **Future Work**

As previously discussed, the primary future work for this project would include the use of live data, using open APIs or proprietary Formula One data feeds depending on the use case available. Additional features can be added in later development cycles, including live alerting, automated strategy suggestions, tyre compound analysis, or predictive modelling.

## References

- [1] V. Hettmann, "Optimizing Race Strategy: A Machine Learning Model for Predicting Formula 1 Pit Stop Timing," Universidade NOVA de Lisboa (Portugal), 2024.
- [2] S. S. W. Fatima and J. Johrendt, "Deep-Racing: An Embedded Deep Neural Network (EDNN) Model to Predict the Winning Strategy in Formula One Racing."
- [3] D. Thomas *et al.*, "Explainable Reinforcement Learning for Formula One Race Strategy," in *Proceedings of the 40th ACM/SIGAPP Symposium on Applied Computing*, 2025, pp. 1090-1097.
- [4] F. Aguad and C. Thraves, "Optimizing pit stop strategies in Formula 1 with dynamic programming and game theory," *European Journal of Operational Research*, vol. 319, no. 3, pp. 908-919, 2024.
- [5] D. Piccinotti, "Open loop planning for formula 1 race strategy identification," 2019.
- [6] O. F. C. Heine and C. Thraves, "On the optimization of pit stop strategies via dynamic programming," *Central European Journal of Operations Research*, vol. 31, no. 1, pp. 239-268, 2023.
- [7] O. F. Carrasco Heine and C. Thraves, "Correction to: On the optimization of pit stop strategies via dynamic programming," *Central European Journal of Operations Research*, vol. 30, no. 4, pp. 1513-1513, 2022.
- [8] B. Msakamali, "F1 data analysis and tactical insights: exploring Formula 1 race performance strategies," 2024.
- [9] M. Leischner, "From Data to Podium: A Machine Learning Model for Predicting Formula 1 Compound Decisions," Universidade NOVA de Lisboa (Portugal), 2023.
- [10] X. Tang, B. Long, and L. Zhou, "Real-time monitoring and analysis of track and field athletes based on edge computing and deep reinforcement learning algorithm," *Alexandria Engineering Journal*, vol. 114, pp. 136-146, 2025.
- [11] A. Li, "Real-Time Athlete Fatigue Monitoring Using Fuzzy Decision Support Systems," *International Journal of Computational Intelligence Systems*, vol. 18, no. 1, p. 23, 2025.
- [12] C. Perin, R. Vuillemot, C. D. Stolper, J. T. Stasko, J. Wood, and S. Carpendale, "State of the art of sports data visualization," in *Computer Graphics Forum*, 2018, vol. 37, no. 3: Wiley Online Library, pp. 663-686.
- [13] K. Eberhard, "The effects of visualization on judgment and decision-making: a systematic literature review," *Management Review Quarterly*, vol. 73, no. 1, pp. 167-214, 2023.

## Appendix A – Project Plan

A separate file, gantt.xlsx, is uploaded alongside the project proposal. The planner can be moved through dates by updating the value in Display Week or using the scrollbar visible above the dates.

|                    |             |              |      |        |
|--------------------|-------------|--------------|------|--------|
| Project Start Date | 26/04/2025  | Display Week | 1    | Week 1 |
| Project Lead       | Harry Adams |              |      |        |
|                    | START       | END          | DAYS | DO     |
| model undercut     | 26/05/2025  | 29/05/2025   | 4    | 0      |

**Display Week**  
Enter the week number to display first in the Gantt Chart. The weeks are numbered starting from the week containing the Project Start Date.